On-Device Object Detection for Assistive Navigation

Zaven Silah
Seaver College
Pepperdine University
Malibu, CA 90263
zaven.silah@pepperdine.edu

Yuki Shen
Seaver College
Pepperdine University
Malibu, CA 90263
yuki.shen@pepperdine.edu

I. Introduction

In 2020, a global meta-analysis published in *Nature* estimated roughly 43.3 million individuals were blind and 295 million lived with moderate or severe vision impairment (MSVI) [1]. Vision loss creates considerable barriers to independent navigation and daily activities. The inability to see is also often associated with reduced quality of life and substantial safety risk. Despite progress in assistive technologies, affordable and scalable tools that deliver real-time environmental awareness remain limited. Advancing a reliable and accessible solution is therefore an essential research area to improve the safety and comfort of individuals with visual impairments.

A. Related Works

Assistive technologies for people with visual impairment have been thoroughly studied across smartphone apps and wearable devices. Common approaches typically adopt existing frameworks such as the YOLO family, MobileNet-SSD, and Deep Convolutional Neural Networks [2]. These frameworks are trained via transfer learning and fine-tuned on the target environment. Then these machine learning models are connected to hardware to provide assistance to an individual. Feedback is usually delivered through verbal hints as well as bone conduction audio, vibration, haptic, and pressure. For instance, Joshi et al. trained a YOLO v3-based on-device detector on a custom annotated dataset that can perform real-time text-to-speech for object names. It is connected to a module with a DSP processor, distance sensor, and camera. The reported average performance is 95.19% detection accuracy [3]. In addition, Masud et al. created a system that provides a verbal description of the object using a TensorFlow Object Detection model connected to a Raspberry Pi and ultrasonic sensors. The paper reports efficient real-time operation with a reported 91% average performance [4].

Although many systems report competitive accuracy and real-time performance, apparent challenges include information overload, limitations on complex environments, constraints on wearable devices, and, critically, the reliance on connectivity. Our work targets an offline, on-device model that maintains real-time operation without network access, emphasizing accessibility to help more individuals. Importantly, we will be the first to integrate distance measurement without introducing additional wearable sensors.

B. Data Sources

Many existing frameworks are pretrained on the COCO (Common Objects in Context) dataset. To improve coverage of street-scene objects, we will apply transfer learning using open-source street-object datasets already in YOLO format, sourced from Roboflow. In addition, evaluation will be based on YouTube's BoundingBoxes dataset combined with real-world environment testing.

C. Specific Problem and Rationale

We prioritize a fully offline, on-device model to avoid dependence on external wearable devices or network connectivity. Eliminating cloud dependency could reduce latency and variability, but it also makes the project harder. Achieving high accuracy under tight computing, memory, and power constraints across a broad array of objects to be included is difficult. Accordingly, the model must be small and optimized for common smartphones or lightweight wearable devices, such as smart glasses or the Raspberry Pi. In the current state, we experimented with multiple open-source, locally processed object detection models. Apple's fastVLM demo excels in recognition but is computing-intensive and limited to iOS, whereas Google's MediaPipe is cross-platform but less accurate. YOLOv11 offers a favorable accuracy-speed trade-off for on-device use, so we will proceed with YOLOv11 as the primary detector.

A more targeted approach based on the aforementioned models may significantly reduce computational impact. Utilizing street-object datasets and COCO may help bridge the gap between larger, more diverse models and those that require less computational power by implementing only specific, important scenes for identification that can be used to identify priority objects in day-to-day commutes, such as vehicles, crosswalks, and road signs.

II. METHODOLOGY

First, our proposed approach implements a cross-platform app built around a lightweight YOLOv11 detector, with local depth processing. Next, we detail the datasets used as a foundation of this project to improve the original YOLO model performance. Finally, our testing protocol combines local evaluation to determine readiness for real-world testing, preceded by field trials across various outdoor conditions.

A. Proposed Approach

Our approach is to develop a cross-platform app for iOS and Android that utilizes YOLOv11 to detect and classify

objects. Specifically, we deploy the smallest detection model, YOLOv11n (around 2.6 million parameters), to minimize latency and enable efficient real-time operation on mobile hardware. The YOLO family excels in performance and compatibility. In our preliminary comparison, intensive models, such as fastVLM, exhibited substantial heat output and degradation in performance within 10 minutes of use, whereas Google's MediaPipe model has noticeably lower recognition accuracy.

To reduce the dependency of our model on any external devices, we aim to use a USB camera to provide input to the mobile device and output audio of the detected objects through Bluetooth headphones. Currently, we will prioritize iOS development due to the availability of hardware. For depth detection, we plan to use Apple's LiDAR APIs for depth detection and output, as well as Google's ARCore depth API in the future to add Android support. ARCore does not depend on LiDAR, which should make it compatible with a broader range of devices. Given the availability and accuracy of LiDAR, we will utilize it for iOS development rather than relying solely on depth computation through images. In terms of hardware, only iOS and Android will be utilized, instead of an external device such as a Raspberry Pi. We want the app to be widely available for use, and many people have a capable phone, but not a Raspberry Pi or the necessary skills to set it up. iOS limits UVC (USB video class) camera support to iPadOS, meaning that iPhones do not support external camera connections via Lightning or Type-C ports. This may limit us to using the built-in camera only for iOS devices, but given that we are also utilizing the built-in LiDAR, finding convenient ways to mount the phone to a user will be our current approach. Android will have the ability to use UVC-mounted cameras, enabling users to keep their phones in their pockets, as LiDAR will not be used for depth detection in that case. We have opted not to utilize depth models like depth-anything-v2, given that LiDAR will perform better. Additionally, Google's ARCore Depth API already uses learning-based monocular depth estimation from motion, which incorporates a neural network on RGB and true depth pairs to fill in ambiguous areas [5]. In the case that LiDAR approximation fails on iPhone, we have integrated Apple's own monocular depth estimation as a fallback measure (but it is less accurate compared to LiDAR).

B. Dataset

Most pre-existing models are trained on the comprehensive COCO dataset. COCO images contain multiple, overlapping objects, which helps the model learn robustness to complicated real-life situations. Because YOLOv11, the model we primarily use, is already trained on COCO, and its original train/test split is unknown, we will not use COCO to test in case of data leakage. Proceeding, we will fine-tune the pre-trained model on the "City Furniture Object Computer Vision Model" [6] and "Potholes Computer Vision Dataset" datasets [7] from Roboflow. These datasets provide additional street-specific labels such as potholes, poles, bus stops, benches, etc, to capture real urban environments. Images are split into train/validation/test sets in a specific YOLO format: for every image file, there's a matching text file serving as labels, each line describes dimension information with all coordinates normalized between 0 and 1 [8]. Additionally, we will use YouTube's BoundingBoxes dataset, which provides a large-scale dataset of videos with human-annotated single-object bounding boxes [9]. Specifically, we will use the dataset titled Object Detection in Video Segments - validation set.

C. Training and Preprocessing

Reading through the Ultralytics (the company that developed YOLO) support documents [10], we realized that when we fine-tune the pre-trained YOLOv11n model with our own data, the detection model is resized and re-initialized for the new class count. This means that although the backbone still benefits from COCO pretraining, the class-prediction layer for the old classes is not preserved, and we are essentially training a new model for our custom dataset. The recommended solution to address this is to retrain the model using the original COCO data plus our custom data.

To achieve the retraining, we do not plan to reuse all 80 object categories, because some object classes - such as a wine glass - are not as relevant to our purpose. We removed some less relevant classes and constructed a subset of COCO that preserves only street-relevant objects (see Appendix I for the exact list of retained categories). Now, we merged the slightly shortened list of COCO training data with the city furniture object and the potholes dataset mentioned above (see Appendix II for the list of added categories across both datasets). The resulting combined set will be applied to retrain the model. This allows the model to retain the vast majority of the original detection ability while learning our new target classes.

We used Ultralytics' own training framework in Python. We specified the optimizer as auto, which, in this case, uses stochastic gradient descent (SGD) by default. We started the training process with 300 epochs, but we stopped the model after 216 epochs when we saw no noticeable improvement.

D. Testing

Our model aims to assist visually impaired users in real-time; most of the testing will take place in realistic environments. Prior to field testing, we will conduct local evaluations on the YouTube BoundingBoxes dataset. As a supervised learning task, evaluation compares predicted detections to ground truth with a confidence level and IoU threshold above a specific set value. From the provided CSV annotations, we first filtered a subset of videos with a sufficient number of bounding box annotations per clip. We then downloaded the corresponding YouTube videos and, for each annotated timestamp, extracted the matching video frame and saved it as an image. We build an index that maps each frame (video ID and timestamp) to its ground-truth class labels. On a laptop, we ran our YOLOv11n model on these extracted frames to obtain predicted bounding boxes, class labels, and confidences. For each prediction, we matched it to at most one ground-truth box of the same class to classify detections as true positives, false positives, or false negatives.

From these matches, we computed per-class precision, recall, and F2-score. The detection performance is reported as a mean Average Precision across all classes with ground-truth annotations. In addition, we will monitor real-time performance metrics relevant to user experience, such as latency. The final composite score will be compared against a preset threshold to decide whether the model is ready for real-world testing.

For the real-life trials, we will evaluate the model on streets and sidewalks by simulating typical user behavior. We will use an annotated checklist of target objects and compare them with model predictions. This test will primarily assess the model's ability to help someone navigate real-life situations, as we anticipate that the model's real-world helpfulness will differ from its quantified evaluation scores.

III. Results

A. On-Device Readiness and Experience

We are able to run the retrained YOLOv11n model locally on an iPhone 16 Pro Max. Using Apple's LiDAR API, we obtain nearly accurate distance estimation using just the iPhone camera. The voice output uses the LiDAR distance measurement and announces detected objects in order from closest to furthest. At this stage, we have limited the model to detect three objects so they can be announced before the next frame is processed. This single-device solution offers a new user experience, with everything handled locally, without an external processing unit or sensors.

After long-term testing, the phone does not appear to experience extreme heat or lag, suggesting the model is computationally reasonable for daily use. Throughout use, the FPS remains constant (30 FPS), and the average latency (the time it takes for YOLO to process the object and for LiDAR to estimate distance) is 14-21ms/detection.

The exact accuracy of the LiDAR distance estimation is hard to quantify in our case. We performed an informal calibration by comparing LiDAR-estimated distances to tape-measure ground truth at several positions. In these tests, the discrepancies were visually small (typically within 1-3 inches), suggesting that our model exhibits distance accuracy consistent with the consensus on LiDAR performance online (for example, an iPhone 16 Pro model has a reported discrepancy of 0.72-3.6 inches [11]). Therefore, for our purposes, incorporating LiDAR is sufficient to provide a distance estimation.

B. Quantified Testing of the Models

We benchmarked the performance difference between the original YOLOv11n model and our fine-tuned model to determine which to deploy for our app. Using the YouTube BoundingBoxes dataset for testing, we compared the performance of both models. Notably, the test set does not include all of our classes. Tailoring a testing set to test per-class optimizations on our fine-tuned model is beyond the scope. The current testing set demonstrates performance on some of the more important classes. The per-class performance matrix for the original YOLO model is shown in Table I, and the per-class performance matrix for the fine-tuned model is shown in Table II. The average mAP for the original model is 0.678, while that for the fine-tuned model is 0.605.

TABLE I

Per-class performance of the original YOLOv11n model.

Class	GT	Precision	Recall	F_2	AP@0.5
bus	19	0.900000	0.947368	0.937500	0.947368
motorcycle	19	0.791667	1.000000	0.950000	0.984636
person	36	0.052764	0.583333	0.193727	0.038551
skateboard	16	0.684211	0.812500	0.783133	0.736364
train	30	0.536585	0.733333	0.683230	0.683440
truck	34	0.666667	0.941176	0.869565	0.925895
umbrella	51	0.340000	0.666667	0.559211	0.431082

TABLE II

Per-class performance of our fine-tuned YOLOv11n model.

Class	GT	Precision	Recall	F_2	AP@0.5
bus	19	0.947368	0.947368	0.947368	0.947368
motorcycle	19	0.904762	1.000000	0.979381	1.000000
person	36	0.066007	0.555556	0.223714	0.040112
skateboard	16	0.714286	0.625000	0.641026	0.500000
train	30	0.869565	0.666667	0.699301	0.646913
truck	34	0.666667	0.941176	0.869565	0.860016
umbrella	51	0.381818	0.411765	0.405405	0.241640

This shows that the two models are actually very similar. The original model slightly outperforms ours, which is expected because we removed many classes, added new labels, and did not address class imbalance in the two Roboflow datasets.

On that topic, the new classes we added to the fine-tuned model are actually not included in the testing. This is because the dataset did not have those labels. But upon real-life testing, we realized that it does not perform well on the newly added classes. This is due to a huge class imbalance: the external datasets combined only have 1269 images, while even the shortened COCO dataset has 90,241 images.

Our fine-tuned model successfully recreated the environment needed to fully retrain and deploy a YOLO model that can replicate its existing classes and learn new ones with additional data. At this time, we decided to deploy the default YOLOv11n model due to its slight performance advantage, as our current fine-tuned test run was unable to reproduce the custom classes. In the future, after further consideration of which classes to keep from YOLO and which to add custom, we will reuse our testing environment to produce a model optimized for classes ideal for outdoor navigation.

C. Real-life Testing of the Model

The most important thing is to conduct real-world testing of the model's performance. At this time, the default YOLOv11n model performs as expected. It detects many desirable classes, such as cars, people, animals, buses, motorcycles, chairs, and more. The depth estimation works very well given LiDAR's accuracy. The voice output needs more fine-tuning; we need to integrate an override function that cuts off the voice when a rapid or more important object is approaching. At this time, it is not user-ready. Given the risks involved in visually impaired navigation, a project of this nature needs further development and may benefit from additional assistive technologies. Nevertheless, as a first-of-

its-kind complete mobile solution, it may reveal promising functionality as the technology develops.

IV. Discussion

Here, we will discuss the problems we encountered while developing the model. Initially, fine-tuning the YOLOv11 model on the city furniture dataset (around 500 images) and the potholes dataset (around 250 images) without the COCO subset yielded poor results. We initialized training with 100 epochs, but the model stopped after around 8 epochs because no meaningful improvement was observed. As shown in figure 1, the precision-recall curves are almost flat near the axes for all classes, and the per-class average precision values (reported in the legend) are extremely low. The aggregated mean average precision over all classes is 0.005, which confirms that this fine-tuning attempt essentially failed: the model does not meaningfully detect new objects, and it does not preserve any of the accuracy from the original training.

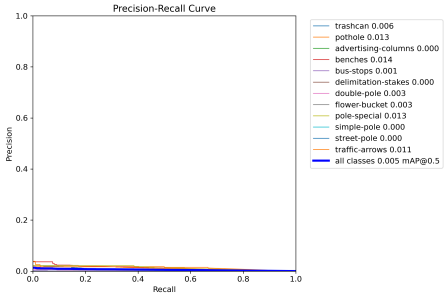


Fig. 1. Precision–recall curves for each custom class on the City Furniture Object and Potholes datasets after fine-tuning YOLOv11n.

Continuing with the topic of training datasets, we initially wanted to use the Mapillary Vistas dataset because it provides real-world street data collected globally. However, YOLO training requires a specific format that did not align with the Mapillary Vistas dataset, and converting it was surprisingly expensive and time-consuming. This is why we resorted to the Roboflow dataset, despite its fewer, lower-resolution images.

In real-world testing, we observed that YOLOv11 simply ignores objects it fails to detect, which is problematic in practice. Intuitively, we attempted to introduce an "other" class that would place bounding boxes over everything. However, this led to unnecessary outputs: the model circled regions of the floor, the wall, or seemingly empty space. In future work, we might experiment with YOLO's instance segmentation model to see whether combining the two can achieve better overall detection without adding computational overhead. Furthermore, the voice output can announce objects in the order from closest to furthest, but it cannot be interrupted if something approaches rapidly. To equip the model for real-world use, we plan to integrate detection of speed or rapid changes in distance, enabling more sophisticated warnings about moving objects. Lastly, future improvements could incorporate a coordination system that provides a general sense of direction and a warning when the person veers off course. This is also achievable with YOLO's oriented detection model that adds an extra angle to locate objects in the image, or the phone's own gyroscope.

Taken together, the results and need for improvement indicate that while our current prototype is not yet ready for unsupervised deployment with visually impaired users, it establishes a complete and reusable pipeline for future development. We demonstrate that an on-device YOLOv11-based detector, combined with distance sensing and audio feedback, can run reliably in real time on consumer hardware without external sensors or cloud support. This framework can be refined by improving training data, class selection, and interaction design, but it already highlights the practical potential of YOLO-based object detection to support safer, independent navigation in everyday environments.

Appendix I
COCO Class Filtering

For reproducibility, we document which COCO classes we retain when constructing our mixed COCO + street-object training set. COCO provides 80 classes (IDs 0–79). We keep the subset in Table below; all other COCO classes are excluded from training.

TABLE III

Subset of COCO classes retained during fine-tuning.

ID	Class name
0	person
1	bicycle
2	car
3	motorcycle
5	bus
6	train
7	truck
9	traffic light
10	fire hydrant
11	stop sign
12	parking meter
13	bench
39	bottle
56	chair
58	potted plant
74	clock

Appendix II
Custom Classes

TABLE IV

Custom classes added to the YOLOv11n model.

ID	Custom class name
0	trashcan
1	pothole
2	advertising-columns
3	benches
4	bus-stops
5	delimitation-stakes
6	double-pole
7	flower-bucket
8	pole-special
9	simple-pole
10	street-pole
11	traffic-arrows
12	traffic-lights-custom
13	traffic-signs-custom
14	other

REFERENCES

[1] R. Bourne *et al.*, "Global prevalence of blindness and vision impairment: a systematic review and meta-analysis," *Nature*, 2024, available at: https://www.nature.com/articles/s41433-024-02961-1.

[2] G. I. Okolo, T. Althobaiti, and N. Ramzan, "Assistive systems for visually impaired persons: Challenges and opportunities for navigation assistance," *Sensors*, vol. 24, no. 11, p. 3572, 2024, available at: https://doi.org/10.3390/s24113572.

[3] R. C. Joshi, S. Yadav, M. K. Dutta, and C. M. Travieso-Gonzalez, "Efficient multi-object detection and smart navigation using artificial intelligence for visually impaired people," *Entropy*, vol. 22, no. 9, p. 941, 2020.

[4] U. Masud, T. Saeed, H. M. Malaikah, F. U. Islam, and G. Abbas, "Smart assistive system for visually impaired people: Obstruction avoidance through object detection and classification," *IEEE Access*, vol. 10, pp. 13 428–13 441, 2022.

[5] Google Developers, "Depth api developer guide (java)," 2025, available at: https://developers.google.com/ar/develop/java/depth/developer-guide.

[6] C. F. Object, "city furniture object dataset," https://universe.roboflow.com/city-furniture-object/city-furniture-object, apr 2024, visited on 2025-12-01. [Online]. Available: https://universe.roboflow.com/city-furniture-object/city-furniture-object

[7] RoadTrain, "Potholes dataset," https://universe.roboflow.com/roadtrain-puq8q/potholes-y1qi8, may 2024, visited on 2025-12-01. [Online]. Available: https://universe.roboflow.com/roadtrain-puq8q/potholes-y1qi8

[8] Ultralytics, "Object detection datasets overview," https://docs.ultralytics.com/datasets/detect/, accessed: 2025-12-01.

[9] E. Real, J. Shlens, S. Mazzocchi, X. Pan, and V. Vanhoucke, "Youtube-boundingboxes: A large high-precision human-annotated data set for object detection in video," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7464–7473. [Online]. Available: https://research.google.com/youtube-bb/

[10] S. S. Rao and B. Q, "Finetuning model," `Ultralytics Community Forum`, July 2025, support discussion thread. [Online]. Available: https://community.ultralytics.com/t/finetuning-model/1252

[11] R. Tamimi, "iphone 16 pro lidar and camera vs survey total station accuracy," YouTube video, 2024, accessed: 2 Dec 2025. [Online]. Available: https://www.youtube.com/watch?v=GObsBqEwq6U